JOSE
The Journal of Open Source Education

Reproducible Machine Learning Workflows for Scientists: A short course in best practices

Matthew Feickert [1]

1 University of Wisconsin–Madison

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Submitted: 14 November 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

A critical component of research software sustainability is the reproducibility of the software and computing environments software is executed in. Researchers need reproducible computing environments for research software applications that may be run across multiple computing platforms — e.g. scientific analyses, visualization tools, data transformation pipelines, and artificial intelligence (AI) and machine learning (ML) applications on hardware accelerator platforms (e.g. GPUs). Modern open source multi-platform environment management tools, e.g. Pixi (Arts et al., 2026; Fischer et al., 2025), provide automatic multi-platform digest-level lock file support for all dependencies — down to the compiler level — of software on public package indexes while still providing a high-level interface well suited for researchers. Combined with the arrival of the full CUDA (Nickolls et al., 2008) stack on conda-forge (conda-forge community, 2015), it is now possible to declaratively specify a full CUDA-accelerated software environment.

This work contains educational resources on applying these aspects of software reproducibility to hardware-accelerated machine learning workflows. The resources are presented as a short course, or "lesson", in The Carpentries (*The Carpentries*, 2026) format designed to be taught to scientific researchers over the course of a three-day workshop. The open source materials are hosted on GitHub by The Carpentries Incubator GitHub organization (Feickert, 2026). The course website is publicly accessible at carpentries-incubator.github.io/reproducible-ml-workflows/.

Statement of Need

Education and training for the broader scientific software community around robust best practices and standards for reproducibility in scientific software is rare, and is practically non-existent for reproducible hardware-accelerated software. This educational material provides tested and validated best practices for creating digest-level reproducible CUDA-accelerated software environments and provides actionable recommendations for adopting computational reproducibility tools. Given the recent advances in enabling technologies, the methods and techniques to support the described best practices and workflows were first made possible in late 2023. As a result, this material is one of the first open source resources available on the topic.

Instructional Design

The course has been designed to be taught over a three-day workshop, with each day divided into a half-day of instruction from the course material, and then a second half-day focusing on applying the course content to real-world research applications to participants' research projects through interactive work sessions with the instructor team. Each "episode" of the

DRAFT

JOSE
The Journal of Open Source Education

workshop course contains interactive "challenge" questions that require participants to put into practice concepts and tools they have just learned, and synthesize information from previous episodes to accomplish tasks. After each challenge question, the instructor team discusses possible solutions with the participants and "live codes" a demonstration of a solution.

Learning Objectives

By the end of the first day of instruction, participants will be able to:

- Describe the terms "reproducibility" and "reuse" as they relate to research computing.
- Describe what is needed for a computational environment to be reproducible.
- Describe what Pixi is, as well as its main features and benefits to reproducible software environments.
- Describe the content and purpose of a lock file.
- Use Pixi's command line interface (CLI) to create a reproducible software environment.
- Describe the structure and functionality of the Pixi manifest file.
- Create and use tasks defined using Pixi's task runner system.
- Create a multi-platform Pixi workspace.

By the end of the second day of instruction, participants will be able to:

- Describe the structure of a conda package in both its `.conda` archive format and its installed form on disk.
- Describe the purpose of a conda virtual package.
- Describe how Pixi platforms that declare a minimum CUDA version enable resolution of CUDA-enabled conda packages.
- Create Pixi software environments that use CUDA-enabled conda packages of machine learning libraries for CUDA-compatible computing platforms.
- Use CUDA-enabled Pixi software environments to train machine learning models on NVIDIA GPUs.

By the end of the third day of instruction, participants will be able to:

- Describe the use of Linux containers for deployment of software environments to remote machines.
- Write a templated Dockerfile that uses Pixi software environments.
- Write a templated Apptainer container image definition that uses Pixi software environments.
- Build a Docker container image that contains a CUDA-accelerated Pixi software environment.
- Build an Apptainer container image that contains a CUDA-accelerated Pixi software environment.
- Write a continuous integration and continuous delivery (CI/CD) configuration file to build and deploy container images to container registries.
- Describe the benefits to the research process of using fully reproducible hardware-accelerated software environments.

If participants are working with access to a high-throughput computing (HTC) facility, they will additionally be able to:

- Write simple HTCondor execution scripts and submit description files that request Linux container and GPU resources.
- Submit distributed machine learning training workflows to HTCondor that run in Linux containers with CUDA-accelerated Pixi software environments.

DRAFT

JOSE
The Journal of Open Source Education

Similarly, if participants are working with access to a high-performance computing (HPC) facility, they will additionally be able to:

- Write simple Slurm execution scripts and batch scripts that request GPU resources.
- Submit distributed machine learning training workflows to Slurm that run in CUDA-accelerated Pixi software environments.

Content

The course is aimed at professional researchers who already have experience with scientific software development and a working understanding of the goals of machine learning. Participants should be familiar with file system structures (e.g. the Filesystem Hierarchy Standard of Unix-like systems), version control software (e.g. Git), and at least one programming language (e.g. Python or C++). Experience with Linux containers and CI/CD systems is not strictly required, but is strongly recommended to gain the most benefit from participation in the course. The focus of the course is on applications using CUDA-enabled Python machine learning libraries, as well as deploying these software environments to production settings on HTC and HPC systems. This course does not teach machine learning concepts, but instead focuses on the methodologies and tools to make existing machine learning workflows reproducible.

Experience of Use

As of 2025, the Reproducible Machine Learning Workflows for Scientists course has been taught to over 70 participants — ranging from university students, to tenured faculty, to industry machine learning researchers from universities, national laboratories, professional organizations, and companies. As the full course content covers multiple concepts and technologies, individual episodes have also been selected to be taught as standalone 4-hour tutorials on targeted topics. Long-term follow-up surveys, querying participants on changes to their scientific workflows three months after participating in a workshop, have shown evidence that participants find Pixi as a technology easy to learn and beneficial enough for researchers that it has changed their normal scientific software workflow habits, becoming a common tool in their regular work. Additional experience teaching the course in different workshop settings is needed to expand the scope of responses to form a more complete understanding of long-term impact on participants.

How the Project Came to Be

This course was motivated by experience and difficulty in creating CUDA-accelerated software environments for machine learning research studies in experimental physics. The challenge was to create environments that could resolve quickly enough to allow for rapid experimentation on local compute resources and then be deployed to remote resources, while also being robust to temporal drift of software versions and compute platform hardware-level system requirements. Work on this multi-year challenge took a new direction in September 2023 (Feickert, 2025) with first experiments in using Pixi and CUDA-enabled PyTorch builds from conda-forge, which led to questions of distributed computing applications for real-world research problems. Further motivation came from conversations with Pixi developers Ruben Arts and Wolf Vollprecht at the 2024 SciPy conference in July 2024. The course content was able to be researched, created, and taught at multiple workshops through support from an Early-Career Fellowship from the US Research Software Sustainability Institute.

DRAFT

JOSE
The Journal of Open Source Education

Acknowledgments

This work was supported by the US Research Software Sustainability Institute (URSSI) via grant G-2022-19347 from the Alfred P. Sloan Foundation. This work benefited from resources and services provided by the University of Wisconsin–Madison Center for High Throughput Computing (Center for High Throughput Computing, 2006), as well as services provided by the OSG Consortium (OSG, 2006, 2015; Pordes et al., 2007; Sfiligoi et al., 2009), which is supported by the National Science Foundation awards #2030508 and #2323298.

References

Arts, R., Zalmstra, B., Vollprecht, W., Jager, T. de, Morcotilo, N., Hofer, J., & Colley, L. (2026). *Pixi*. https://github.com/prefix-dev/pixi/releases/tag/v0.84.1

Center for High Throughput Computing. (2006). *Center for high throughput computing*. Center for High Throughput Computing. https://doi.org/10.21231/GNT1-HW21

conda-forge community. (2015). *The conda-forge Project: Community-based Software Distribution Built on the conda Package Format and Ecosystem*. https://doi.org/10.5281/zenodo.4774216

Feickert, M. (2025). *Distributing your science: Turning analyses into scientific tools* (Version v0.0.2). Zenodo. https://doi.org/10.5281/zenodo.17606840

Feickert, M. (2026). *Reproducible machine learning workflows for scientists*. Zenodo. https://doi.org/10.5281/zenodo.17537698

Fischer, T., Vollprecht, W., Zalmstra, B., Arts, R., Jager, T. de, Fontan, A., Hines, A. D., Milford, M., Traversaro, S., Claes, D., & Raine, S. (2025). *Pixi: Unified software development and distribution for robotics and AI*. https://arxiv.org/abs/2511.04827

Nickolls, J., Buck, I., Garland, M., & Skadron, K. (2008). Scalable parallel programming with CUDA. *ACM SIGGRAPH 2008 Classes*. https://doi.org/10.1145/1401132.1401152

OSG. (2006). *OSPool*. OSG. https://doi.org/10.21231/906P-4D78

OSG. (2015). *Open science data federation*. OSG. https://doi.org/10.21231/0KVZ-VE57

Pordes, R., Petravick, D., Kramer, B., Olson, D., Livny, M., Roy, A., Avery, P., Blackburn, K., Wenaus, T., Würthwein, F., Foster, I., Gardner, R., Wilde, M., Blatecky, A., McGee, J., & Quick, R. (2007). The open science grid. *J. Phys. Conf. Ser.*, *78*, 012057. https://doi.org/10.1088/1742-6596/78/1/012057

Sfiligoi, I., Bradley, D. C., Holzman, B., Mhashilkar, P., Padhi, S., & Wurthwein, F. (2009). The pilot way to grid resources using glideinWMS. *2009 WRI World Congress on Computer Science and Information Engineering*, *2*, 428–432. https://doi.org/10.1109/CSIE.2009.950

The Carpentries. (2026). https://carpentries.org/.

DRAFT